SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1702**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 50%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I _Vijiya Priya S.A (192524417)_(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Vijiya Priya S.A
192524417

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Question 1: AI Drones for Emergency Medicine Delivery (Greedy Best-First Search vs A*)

1.1 Problem formulation

The region is modelled as a weighted graph. States are waypoints (grid cells or landmarks); the initial state is the drone base; the goal is the delivery point; actions are moves between neighbouring waypoints; the step cost g(n) depends on distance, terrain and weather (wind, rain, flooding); and the heuristic h(n) is the straight-line (Euclidean) distance to the goal. Because the straight line ignores obstacles and weather, it is an optimistic estimate and is therefore admissible and consistent.

1.2 Behaviour of Greedy Best-First Search (GBFS)

GBFS expands the node with the smallest heuristic value, f(n) = h(n). In the flood scenario the drone simply keeps heading to whichever waypoint looks closest to the hospital in a straight line, ignoring what it has already spent.

Strengths: very fast and memory-light; expands few nodes when the terrain is open; produces an immediate direction of travel, which suits urgent decisions; needs no accurate cost model.

Weaknesses under uncertainty: it is neither optimal nor complete in graphs with cycles unless a visited list is kept; it can be lured into dead ends (a river or collapsed bridge directly between the drone and the goal looks "close" but is blocked); it ignores weather-dependent costs, so it may choose a short path through a storm cell; and a single wrong heuristic estimate can send it far off course with no mechanism for recovery other than re-planning.

1.3 Behaviour of A* Search

A* uses f(n) = g(n) + h(n), where g(n) is the actual accumulated cost (distance + terrain + weather penalty) and h(n) is the estimated remaining cost. The g(n) term keeps the search honest about the cost already paid, while h(n) pulls the search toward the goal. Early on, h dominates and A* behaves like a directed search; when a region proves expensive (strong winds, flooded terrain), g(n) grows and A* automatically abandons that frontier and expands cheaper alternatives. With an admissible heuristic A* is complete and optimal, so the delivered route has minimum total delay/risk cost given the current map.

How the balance works: if h(n) = 0, A* degenerates into Uniform-Cost Search (optimal but slow); if g(n) = 0, it becomes GBFS (fast but unsafe). A* sits between the two. Weighted A* ($f = g + w \cdot h$ with w slightly above 1) allows the designer to trade a small amount of optimality for much greater speed.

1.4 Impact of heuristic accuracy

Heuristic quality	Effect on Greedy Best-First	Effect on A*
Accurate (close to true cost)	Goes almost straight to the goal; good paths and very few expansions.	Expands almost only nodes on the optimal path; optimal and very efficient.
Underestimates heavily (e.g., straight line in a mountainous, flooded area)	Misled by false closeness; wanders into traps and may revisit regions.	Still optimal (admissible) but expands many nodes, becoming slower.
Overestimates (inadmissible)	Little change in character; still guided purely by h.	Loses optimality guarantee; may return a longer or riskier route.
Stale / noisy (due to updates)	Quality degrades sharply because h is the only signal.	Degrades gracefully because g(n) anchors decisions in real cost.

In short, GBFS is far more sensitive to heuristic errors, whereas A* only pays a performance penalty. A better heuristic can be built by adding a weather/terrain multiplier to the straight-line distance, while keeping it admissible.

1.5 Effect of dynamic changes (blocked paths)

When a path is suddenly blocked, the previously computed plan becomes invalid. GBFS can recover quickly because it simply chooses the next-best h node, but it has no memory of cost, so the new path may be much worse. Plain A* must recompute from the drone's current position, which can be expensive when many edges change. The practical remedy is **incremental or anytime replanning**: D* Lite or Lifelong Planning A* reuse earlier search effort and only repair the affected portion of the search tree. Replanning is triggered whenever new information about blocked edges arrives, the blocked edge is assigned infinite cost, and the search resumes from the drone's current location. Search performance (nodes expanded,

replanning time) falls as the rate of change increases, so the system should replan locally within a limited horizon and keep a few alternative routes cached.

1.6 Recommendation

A* (preferably in its incremental form, D* Lite / Weighted A*) is the most suitable algorithm. Justification**

v. **Real-time constraints:** with an informative heuristic A* is fast enough; Weighted A* or an anytime variant gives a first feasible route quickly and improves it if time remains.

vi. **Optimality:** A* returns the minimum-cost route under the known cost model, minimising delay, energy use and exposure to bad weather.

vii. **Safety:** by including risk (storm zones, no-fly areas, low battery margin) in g(n), A* avoids dangerous shortcuts that GBFS would take; blocked paths are handled by replanning.

GBFS may be used only as a fallback emergency mode when computing time is nearly zero or the drone must immediately choose a heading.

Question 2: Smart City Traffic Signal Optimization (Local Search)

> *Scenario: An AI system tunes signal timings across hundreds of intersections to minimise total waiting time, fuel consumption and congestion. The search space is huge, traffic is dynamic, and the system may get stuck at locally optimal solutions.*

2.1 Formulation as an optimization problem

Element	Definition
Decision variables	For each intersection i: cycle length C_i, green split for each phase g_ij, and offset o_i relative to neighbours (to create "green waves").
State	A complete assignment of timings to all intersections.
Objective function	Minimise F = w1·(total waiting time) + w2·(fuel consumption) + w3·(congestion index/queue length), where the weights express city priorities.
Constraints	Minimum green time for pedestrians, maximum cycle length, emergency-vehicle priority, fairness (no approach starved).
Neighbourhood	Change one variable slightly (e.g., ±2 s green at one intersection) or shift one offset.

Why traditional (systematic) search is inefficient: if each of N = 300 intersections has even 10 candidate timing plans, there are 10^300 states, so BFS, DFS, uniform-cost or even A* cannot enumerate or store them. The problem is a *configuration* problem where the path to the solution is irrelevant; only the final state matters. Evaluating a state needs a traffic simulation, which is costly. Moreover, intersections are strongly coupled (changing one alters flow in its neighbours), and traffic patterns change every few minutes, so an exact optimal solution would be obsolete before it is found. Local search, which keeps only a single current state, uses constant memory and can give good solutions at any time.

2.2 Hill Climbing and its limitations

Hill Climbing starts from the current timing plan (or a random one) and repeatedly moves to the best neighbouring plan that lowers F (steepest-ascent variant) or to the first improving neighbour (first-choice variant). It stops when no neighbour is better. For traffic control this means: adjust one signal, simulate or measure the effect, keep the change if waiting time decreases, and repeat.

viii. **Advantages:** simple, quick, low memory, can run continuously and adapt online.

ix. **Limitations:** incomplete and non-optimal; gets trapped in local optima, plateaus and ridges; its result depends heavily on the starting plan; it never accepts a temporary worsening, so it cannot escape; and in a dynamic setting it can lock onto a plan that suits yesterday's traffic.

2.3 Local maxima, plateaus and ridges: real-world interpretation

Problem	Meaning in search	Traffic interpretation
Local maximum (minimum of cost)	A state better than all neighbours but worse than the global best.	Every single-signal tweak increases delay, yet a coordinated change across a whole corridor would greatly reduce it. The city is "stuck" with a mediocre but stable plan.
Plateau	A flat region where neighbours have equal value.	Changing green time by 1–2 s at one junction has no measurable effect on total waiting time, so the algorithm cannot tell which direction is better and wanders.

Problem	Meaning in search	Traffic interpretation
Ridge	A narrow path to the optimum that single moves cannot follow because they pass through worse states.	Improving the green wave needs several adjacent intersections to change their offsets simultaneously; changing one at a time breaks the synchronisation and makes things worse.

Typical remedies are random restarts, sideways moves with a limit, stochastic hill climbing, and first-choice hill climbing, but these only partly solve the problem.

2.4 Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA) picks a random neighbour. If it improves F it is accepted; if it worsens F by ΔE it is still accepted with probability e^(−ΔE/T). The temperature T starts high (exploring broadly, accepting many bad moves) and is slowly lowered by a cooling schedule, so the search gradually becomes greedy. This lets the system climb out of local optima, cross plateaus and move along ridges. It still uses a single state, so it is memory-efficient, and it is **asymptotically optimal** with a sufficiently slow schedule. In traffic control, a brief increase in delay at one junction may be accepted if it later unlocks a better corridor-wide coordination.

Genetic Algorithms (GA) maintain a *population* of candidate timing plans (chromosomes encoding all cycle lengths, splits and offsets). Each generation: (i) evaluate fitness = 1/F using a simulator; (ii) select the fittest plans; (iii) perform **crossover** (combine the plans of different regions, so a good corridor from one plan joins a good corridor from another); (iv) apply **mutation** (random timing changes for diversity); and (v) replace the old population. GA explores many regions in parallel, naturally combines good partial solutions, and is easy to parallelise across servers or districts.

2.5 Recommendation

A hybrid approach is recommended: a Genetic Algorithm for city-wide, periodic (e.g., hourly or per-pattern) global optimisation, followed by Simulated Annealing / hill climbing for fast local fine-tuning every few minutes. Justification:

x. **Scalability:** GA evaluates populations in parallel, and the city can be decomposed into zones optimised separately and then merged; SA needs only one state and little memory.

xi. **Adaptability:** the previous best population or solution seeds the next run (warm start), so the system responds quickly when traffic patterns change; SA with a re-heated temperature handles sudden events such as accidents or matches.

xii. **Solution quality:** both methods avoid the local-optimum trap of pure hill climbing and yield near-optimal, robust plans, which is all that is required in a continuously changing environment.

Question 3: Autonomous Mars Rover (Online Search)

> *Scenario: A rover explores unknown Martian terrain, must reach sample sites safely, avoid craters and rocks, has limited sensors and no complete map, and updates its knowledge as it explores.*

3.1 Why an online search agent is required

An **offline** agent computes a complete plan before acting and then executes it blindly; it assumes full knowledge of the state space, actions and costs. The rover does not have such a map. In **online search**, the agent interleaves computation and action: it acts, observes the percepts, updates its model, and then decides the next step. This is essential because:

xiii. the environment is **unknown** (exploration problem): the results of actions and even which actions are possible can only be discovered by trying them;

xiv. communication with Earth has a delay of 4–24 minutes each way, so remote step-by-step control is impossible and the rover must decide autonomously;

xv. a pre-computed plan could lead the rover into an unseen crater or rock field; acting step by step with fresh sensor data keeps the rover safe;

xvi. computation is limited by on-board power and processing, so planning the whole route in advance is wasteful when it will probably change.

3.2 Challenges: partial observability and dynamic environments

xvii. **Limited sensing range:** cameras and LIDAR only see nearby terrain; shadows, dust and slope can hide hazards, so the agent works with a belief state rather than a full state.

xviii. **Sensor noise and localisation drift:** wheel slip on loose sand and imperfect odometry cause errors in position estimates, so SLAM (simultaneous localisation and mapping) is required.

xix. **Irreversible actions / dead ends:** a step into a deep crater or a wheel stuck in sand cannot be undone; the agent must consider the safety of exploration (an "unsafe" or non-reversible state can make the goal unreachable).

xx. **Dynamic changes:** dust storms, shifting dunes, changing light and temperature, and power or battery limits alter traversability and cost over time.

xxi. **Competitive ratio:** online algorithms are judged against what an offline agent with a full map would have achieved; in the worst case an adversarial environment can make the online cost arbitrarily larger, so conservative strategies are required.

3.3 Building and updating the internal model

The rover maintains a **map (model) M** of visited states: for every explored state s and action a it stores the resulting state s' = result(s, a) and the observed cost. The cycle is:

xxii. **Sense:** collect percepts from cameras, LIDAR, IMU and wheel encoders.

xxiii. **Update:** add newly observed cells to an occupancy grid with traversability scores (free / rock / crater / unknown); update the cell probabilities using Bayesian filtering; correct the pose using landmarks (SLAM).

xxiv. **Plan:** run a local search (e.g., A* or D* Lite) on the current map towards the target or nearest frontier, treating unknown cells with a cautious cost.

xxv. **Act:** move one or a few steps, then repeat. If a planned step is found to be blocked, mark it as an obstacle and replan.

Typical online algorithms are **online DFS** (with backtracking to the most recent unexplored predecessor, needs reversible actions), **LRTA*** (Learning Real-Time A*, which stores and updates h(s) values of visited states so that the heuristic improves with experience and the agent escapes local minima), and **D* Lite** for incremental replanning.

3.4 Online search vs uninformed and informed search

Aspect	**Uninformed (BFS/DFS/UCS)**	**Informed (A*, Greedy) – offline**	**Online search (LRTA*, online DFS, D* Lite)**
Knowledge needed	Full model of states and actions	Full model + heuristic	Only the current percept and what has been discovered

Aspect	Uninformed (BFS/DFS/UCS)	Informed (A*, Greedy) – offline	Online search (LRTA*, online DFS, D* Lite)
When does it act?	After the full plan is found	After the full plan is found	Interleaves planning and acting
Handles unknown terrain	No	No (needs a map)	Yes
Efficiency	Poor: explores blindly	Good: heuristic guides the search	Good: heuristic + learning from experience
Safety / replanning	No replanning	Must restart on change	Natural replanning after every observation
Optimality	BFS/UCS optimal (given a model)	A* optimal (given a model)	Not guaranteed; judged by competitive ratio

3.5 Most suitable approach and justification

An online, informed, replanning approach is recommended: LRTA* / D* Lite combined with an occupancy-grid map, hazard-aware cost functions and frontier-based exploration. The rover moves toward its sample target using an informed heuristic (distance to goal), treats unknown cells as moderately risky, and replans immediately after each sensor updat**

xxvi. **Uncertainty:** the belief map and Bayesian updates represent what is unknown; a safety margin around craters/rocks and cautious costs for unknown cells reduce risk.

xxvii. **Adaptability:** D* Lite reuses previous computation and quickly repairs the path when new obstacles appear; LRTA* improves its heuristic with each visit.

xxviii. **Safety:** only actions into verified safe or reversible states are preferred; hard constraints on slope, tilt and battery reserve trigger a retreat to a known safe position when the risk is high.

Question 4: University Exam Timetabling (Constraint Satisfaction Problem)

> *Scenario: A university needs to automatically generate exam timetables with no overlapping exams for any student, limited exam halls, precedence between subjects, faculty availability, and special accommodations; complexity grows with courses and students.*

4.1 Formulation as a CSP

A CSP is defined by a triple <X, D, C>: a set of variables X, a domain D_i for each variable, and a set of constraints C that restrict the combinations of values. Here the goal is to assign every exam to a time slot and a hall so that all constraints hold (a *consistent and complete* assignment).

4.2 Variables, domains and constraints

Component	**Definition and explanation**
Variables	For each course exam E_i: **Slot_i** (time) and **Hall_i** (room). Optionally an invigilator variable Inv_i. If there are 60 courses, there are 120–180 variables.
Domain of Slot_i	All available (day, session) pairs, e.g., {D1-AM, D1-PM, D2-AM, ..., D10-PM}. Domains are reduced by unary constraints (faculty not available, holidays).
Domain of Hall_i	Halls whose capacity ≥ enrolled students of E_i, e.g., {H1, H2, ..., H8}; special halls for accommodations.
C1: No student clash	If courses E_i and E_j share at least one student, then Slot_i ≠ Slot_j (binary constraint).
C2: Hall capacity / exclusivity	If Slot_i = Slot_j then Hall_i ≠ Hall_j (one hall hosts one exam per slot, unless exams are allowed to share a large hall with enough seats); number of exams in a slot ≤ number of halls.
C3: Precedence	If subject A must be before B: Slot_A < Slot_B (ordering constraint).
C4: Faculty availability	Slot_i ∈ available slots of the faculty/invigilators of E_i (unary), and an invigilator cannot supervise two simultaneous exams.
C5: Special accommodations	Students needing extra time/separate rooms: assign E_i to rooms with the necessary facilities and reserve longer slot windows; no back-to-back exams for them.

Component	Definition and explanation
Soft constraints (optional)	Avoid more than two exams for a student in a day; spread exams evenly; minimise total exam period. Used for ranking among valid solutions.

The constraint graph has an edge between two exam variables whenever the two courses share a student, so it resembles graph colouring (slots = colours), a well-known NP-complete problem; this is why scalability needs careful treatment.

4.3 Backtracking search

Backtracking search is a depth-first search that assigns one variable at a time and checks constraints after each assignment; if a violation occurs, it undoes (backtracks) the last assignment and tries another value. The steps are:

xxix. Select an unassigned variable (e.g., exam "AI" first).

xxx. Choose a value from its domain (e.g., D1-AM, H1) and check consistency with already assigned variables.

xxxi. If consistent, recurse to the next variable; if no value works, backtrack to the previous variable and try its next value.

xxxii. Stop when all variables are assigned (solution) or all possibilities are exhausted (no solution).

For example, if the AI exam and the DBMS exam share students and AI has been assigned D1-AM, DBMS cannot take D1-AM. If later a Networks exam finds no slot left (all slots conflict), the algorithm backtracks to change the earlier assignments. Pure backtracking is complete but can show *thrashing*: it may repeat the same failure many times deep in the tree, with a worst case of O(d^n).

Its performance is improved with heuristics: **MRV** (Minimum Remaining Values: choose the variable with the fewest legal values, e.g., an exam with limited faculty availability), the **degree heuristic** (choose the exam with the most conflicts with other exams), and **LCV** (Least Constraining Value: choose the slot that leaves most options for the neighbours).

4.4 Constraint propagation and forward checking

Forward checking removes inconsistent values from the domains of unassigned neighbours immediately after each assignment. When the AI exam is assigned D1-AM, "D1-AM" is deleted from the domain of every course that shares students with AI; if any domain becomes

empty, the algorithm backtracks at once, long before reaching that variable (early failure detection). **Arc consistency (AC-3)** goes further: for every pair (X_i, X_j), a value in D_i is kept only if some value in D_j supports it, and these reductions are propagated through the network. MAC (Maintaining Arc Consistency) runs AC-3 after every assignment.

Technique	Benefit	Cost
Plain backtracking	Simple; low overhead per node	Late detection of failure; many wasted branches
Forward checking	Detects dead-ends early; prunes domains; cheap	Does not detect all inconsistencies between unassigned variables
Arc consistency / MAC	Strongest pruning; far fewer nodes expanded	Higher time per node; worthwhile for tightly constrained problems

4.5 Real-world challenges and recommended strategy

xxxiii. **Scale:** thousands of courses and students create dense constraint graphs and a huge search space; the problem is NP-hard.

xxxiv. **Over-constrained instances:** it may be impossible to satisfy every constraint (e.g., too few halls or conflicting accommodations), requiring soft constraints and priorities.

xxxv. **Late changes:** course withdrawals, hall unavailability or faculty changes require re-solving quickly with minimal disruption to the published timetable.

xxxvi. **Fairness and quality:** a feasible timetable is not necessarily a good one (student fatigue, gaps between exams).

Recommended strategy: model the problem as a CSP and solve it using backtracking with **MRV + degree + LCV heuristics, forward checking/MAC, and conflict-directed backjumping**, preceded by preprocessing (node and arc consistency) and by decomposing the problem (e.g., by department or by tree-structured parts). For very large instances, combine this with **local search (min-conflicts)** to repair an initial assignment, and use soft-constraint optimisation to improve quality. For late changes, only re-assign the affected exams (incremental repair). This hybrid scales better than naive backtracking and keeps timetables valid, fair and easy to update.

Question 5: Strategic Game AI (Minimax and Alpha-Beta Pruning)

> *Scenario: A gaming company builds an AI opponent for a real-time strategy game. The AI competes against humans, must decide within strict time limits, evaluate complex states with many moves, and the decision tree is extremely large.*

5.1 How Minimax models the problem

The game is modelled as an adversarial search problem: **states** (positions of units, resources, buildings), **players** (MAX = the AI, MIN = the human), **actions** (build, move, attack, gather), a **transition model**, a **terminal test** (base destroyed, time out) and a **utility function** (win = +1, loss = −1, draw = 0, or a score). Minimax assumes that both players play optimally: MAX picks the move that **maximises** the value, and MIN replies with the move that **minimises** it. Values are computed recursively from the leaves upwards:

MINIMAX(s) = UTILITY(s) if s is terminal; max over actions of MINIMAX(RESULT(s,a)) if it is MAX's turn; min over actions of MINIMAX(RESULT(s,a)) if it is MIN's turn.

It is complete (for finite trees) and optimal against an optimal opponent, with time complexity O(b^m) and space complexity O(bm) when implemented depth-first, where b is the branching factor and m is the maximum depth.

5.2 Computational challenges of large game trees

xxxvii. **Combinatorial explosion:** RTS games may have 50–1000 legal moves per turn (b), so searching m = 10 plies yields about 50^10 ≈ 10^17 nodes, far beyond any time limit.

xxxviii. **Real-time constraint:** the AI may have only 50–200 ms per decision, so a complete search to terminal states is impossible; a cut-off depth is needed.

xxxix. **Imperfect information and chance:** fog of war, simultaneous moves and random events violate the assumptions of pure Minimax.

xl. **Expensive state evaluation and memory:** each node requires copying/simulating a complex game state; repeated states (transpositions) waste effort.

xli. **Non-optimal opponents:** humans are not perfect, so assuming optimal play can be overly pessimistic.

5.3 How Alpha-Beta pruning improves efficiency

Alpha-Beta pruning computes the **same Minimax value** while skipping branches that cannot influence the final decision. It maintains two bounds: **α** = the best (highest) value MAX can already guarantee along the path, and **β** = the best (lowest) value MIN can already guarantee. At each node, if $\alpha \geq \beta$, the remaining children are pruned (a cutoff).

Reasoning with an example: suppose MAX has examined its first move and found it guarantees 6 ($\alpha = 6$). While evaluating the second move, MIN finds a reply that gives 4. Since MIN will choose at most 4 here, which is worse than 6 for MAX, MAX will never choose this second move, so the other replies under it need not be evaluated at all. The whole subtree is cut off without affecting the result.

Property	**Result**
Correctness	Returns exactly the same move and value as full Minimax; pruning never changes the answer.
Best-case complexity	With perfect move ordering: O(b^(m/2)), so the effective branching factor becomes √b. The search can reach **twice the depth** in the same time.
Average case	With random ordering, about O(b^(3m/4)).
Worst case	With the worst ordering no pruning occurs: O(b^m).
Move ordering	Try promising moves first (captures, attacks on key buildings, moves from the previous best / killer moves / history heuristic) to maximise cutoffs.
Extra optimisations	Transposition tables (cache evaluated states), iterative deepening (always have a best-so-far move when time expires), and aspiration windows.

5.4 Design of an evaluation function

Because the search is cut off at a limited depth, non-terminal states must be scored with a heuristic evaluation function that estimates the expected utility. A common design is a weighted linear combination:

$$Eval(s) = w1 \cdot f1(s) + w2 \cdot f2(s) + \ldots + wn \cdot fn(s)$$

Feature f_i	Description	Why it matters (sample weight)
Material / army strength	Weighted count of own units minus enemy units (by cost and combat power)	Direct measure of fighting capability (w = 0.30)
Economy and resources	Resource income rate, stockpile, number of workers/expansions	Long-term capacity to produce units (w = 0.20)
Map and position control	Territory controlled, control of choke points, vision	Mobility and ability to attack or defend (w = 0.15)
Technology / upgrades	Research level, availability of advanced units	Future strength (w = 0.10)
Base safety and threats	Health of the main base, enemy units near the base, vulnerability	Protects against sudden loss (w = 0.15)
Tempo / initiative	Production queue, unit mobility, attack opportunities	Ability to dictate the game (w = 0.10)

Design principles: the function must be fast to compute (it runs at every leaf), must correlate strongly with the actual chance of winning, should be applied only to *quiescent* (stable) positions to avoid the horizon effect, and should return extreme values for won/lost terminal states. The weights can be set from expert knowledge or learned from game records through machine learning (e.g., regression, self-play, reinforcement learning) and then tuned against test opponents.

5.5 Strategy balancing optimality and real-time performance

A pure optimal Minimax search is infeasible, so the recommended strategy is a **time-bounded, depth-limited Alpha-Beta search** with the following components:

xlii. **Iterative deepening with a time manager:** search depth 1, 2, 3, ... and stop when the time budget expires, always returning the best move of the last completed iteration.

xliii. **Good move ordering + transposition table + killer/history heuristics** to maximise pruning and avoid repeated work.

xliv. **Selective search:** prune obviously weak moves with forward pruning/beam search, extend critical lines (threats, attacks), and use abstraction of the game state (grouping units, coarse time steps) to reduce the branching factor.

xlv. **Quality evaluation function** (Section 5.4), with quiescence search at the horizon.

xlvi. **Anytime and hybrid approaches:** run the search in a background thread so that the decision is available whenever needed; for very large or uncertain situations, combine with Monte Carlo Tree Search (MCTS) or scripted openings and tactics.

This approach gives the AI a strong, near-optimal decision in the available time while remaining responsive; and the difficulty level can be adjusted by limiting the depth or time, so human players find the opponent challenging but fair.

Summary of Recommended Approaches

Q	Scenario	Recommended technique	Key reason
1	Emergency drone delivery	A* (incremental/weighted, D* Lite)	Optimal and safe; replans when paths are blocked
2	Traffic signal optimization	Genetic Algorithm + Simulated Annealing	Escapes local optima; scalable and adaptive
3	Mars rover exploration	Online search (LRTA*, D* Lite) with map building	Works with an unknown, partially observable environment
4	Exam timetabling	CSP: backtracking + MRV/LCV + forward checking/MAC	Prunes early; hybrid with min-conflicts for scale
5	Real-time strategy game AI	Depth-limited Alpha-Beta + iterative deepening + heuristic evaluation	Near-optimal decisions within strict time limits

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand